Project

N-QUEENS PROBLEM

Constraints Satisfaction Problem



PROJECT REPORT FILE

Subject: Artificial Intelligence

Subject Code: BTAIML502-20

Submitted To:

Dr. Navleen Singh Rekhi

Assistant Professor

Submitted By:

406/24: Abhi Saurav Saroya

437/24: Gagandeep

441/24: Gautam

BTech CSE (AI-ML) Sec-5(A)

N-QUEENS PROBLEM

Aim: To study and implement the N-Queens problem using Python by placing N queens on an N × N chessboard such that no two queens attack each other, while satisfying all the constraints of the problem.

Apparatus (Software Requirements): Python 3.x, Code Editor or IDE, Python Interpreter

Theory: The N-Queens problem is a classical combinatorial problem in Artificial Intelligence. It involves placing N queens on an N × N chessboard such that no two queens attack each other. The problem is widely used to demonstrate search algorithms, constraint satisfaction, backtracking and optimization techniques. It is an example of a problem where a large number of possible arrangements must be examined to identify a valid solution satisfying a given set of constraints.

Rules of the N-Queens Problem: A queen in chess can attack other pieces horizontally, vertically and diagonally.
Therefore, a valid arrangement of N queens must satisfy three main rules:

1. Row Constraint: Only one queen can be placed in each row.
2. Column Constraint: Only one queen can be placed in each column.
3. Diagonal Constraint: No two queens can occupy the same diagonal.

If any two queens violate these conditions, the arrangement is considered invalid.

Representation of the Chessboard: The chessboard can be represented using a two-dimensional array or matrix of size N × N.
Each cell represents a position on the chessboard.

- 0 represents an empty cell.
- 1 represents a cell occupied by a queen.

For example, a 4 × 4 chessboard can be represented as:

```
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
```

This arrangement represents a valid solution to the 4-Queens problem.

Pseudocode:

```
START

1. Read the value of N.

2. If N is less than or equal to 0:
        Display "Invalid board size".
        STOP.

3. Initialize an array to store the column position of each queen.
```

4. Define a function to check whether a queen can be safely placed at a given position.

5. Check whether the position violates any previously placed queen's:
    a. Column constraint.
    b. Diagonal constraint.

6. If the position is safe:
    a. Place the queen.
    b. Move to the next row.
    c. Recursively repeat the process.

7. If no valid position is available:
    a. Backtrack to the previous row.
    b. Remove the previously placed queen.
    c. Try another position.

8. If all N queens are successfully placed:
    Display the chessboard arrangement.

9. Otherwise:
    Display "No solution exists".

STOP

Source Code:

```
def is_safe(board, row, col):
    for previous_row in range(row):
        previous_col = board[previous_row]
        if previous_col == col:
            return False

        if abs(previous_col - col) == abs(previous_row - row):
            return False

    return True

def solve_n_queens(n):
    board = [-1] * n

    def place_queen(row):
        if row == n:
            return True

        for col in range(n):
            if is_safe(board, row, col):
                board[row] = col
                if place_queen(row + 1):
                    return True
                board[row] = -1
        return False
```

```
    if place_queen(0):
        return board

    return None

def display_board(board, n):
    print("\nSolution:")
    for row in range(n):
        for col in range(n):
            if board[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()

# Main program
n = int(input("Enter the number of queens (N): "))

if n <= 0:
    print("Invalid input! N must be a positive integer.")
else:
    solution = solve_n_queens(n)
    if solution is not None:
        display_board(solution, n)
    else:
        print("No solution exists for N =", n)
```

Sample Output:

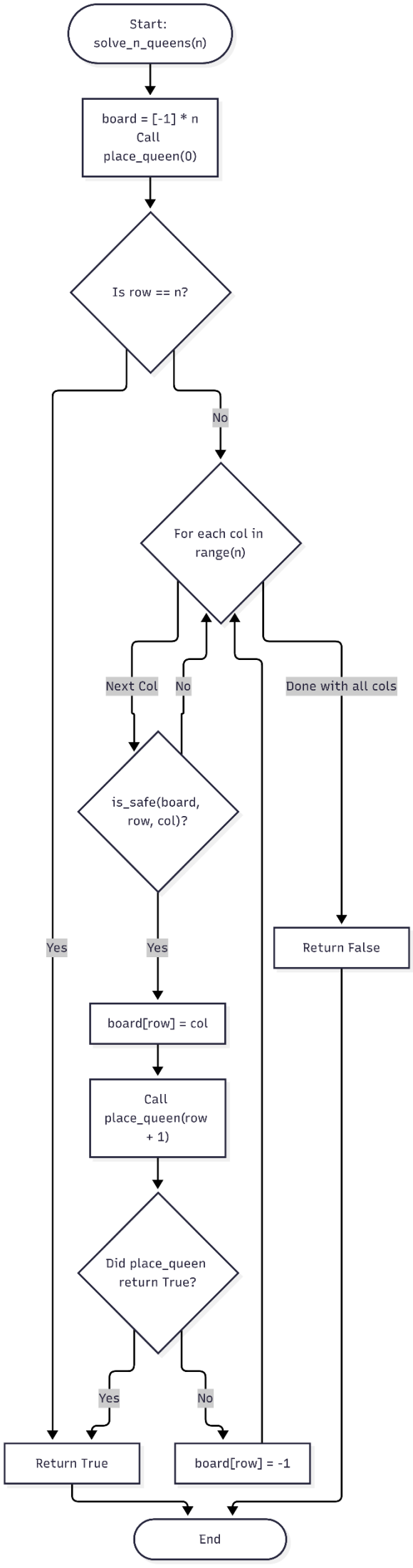
```
Enter the number of queens (N): 8

Solution:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

```
Enter the number of queens (N): 3
No solution exists for N = 3
```

```
Enter the number of queens (N): -7
Invalid input! N must be a positive integer.
```

Flowchart:


Start: solve_n_queens(n)
board = [-1] * n Call place_queen(0)
Is row == n?
No
For each col in range(n)
Next Col
No
Done with all cols
is_safe(board, row, col)?
Yes
Yes
Return False
board[row] = col
Call place_queen(row + 1)
Did place_queen return True?
Yes
No
Return True
board[row] = -1
End

Result: The N-Queens problem was successfully implemented using Python and the Backtracking algorithm. The program accepts the value of N, checks the constraints for queen placement, and finds a valid arrangement in which no two queens attack each other. The program successfully handles valid board sizes, and cases where no solution exists.

Conclusions: The N-Queens problem was successfully solved using the Backtracking technique in Python. The practical helped in understanding state-space search, constraint checking and recursive problem-solving in Artificial Intelligence.

Pitfalls and Challenges Faced:

1. Attacking Queens: Placing multiple queens in the same row, column or diagonal could result in an invalid arrangement. This was overcome by implementing a safety-checking function before placing each queen.
2. Unnecessary Search: Exploring every possible arrangement without checking constraints could increase computational effort. This was overcome by rejecting unsafe positions immediately.
3. Dead-End Situations: A queen placement could prevent the remaining queens from being placed. This was overcome using backtracking, which removes the previous queen and explores alternative positions.
4. Unsolvable Board Sizes: Certain values of N do not have a valid solution. This was overcome by checking the result of the search algorithm and displaying an appropriate message when no solution exists.

Overall Conclusion: The practical demonstrated how Backtracking can systematically explore possible solutions while eliminating invalid arrangements. It provided a clear understanding of how Artificial Intelligence techniques can solve constraint-based problems efficiently.